

Client Segmentation Platform

Documentación Técnica del Sistema

Repositorio: product/ — Backend FastAPI + Frontend React

Índice de contenidos

1. Arquitectura y tecnologías utilizadas
2. Métodos de segmentación implementados
3. Tratamiento de valores nulos y decisiones de calidad de datos
4. Dashboards y visualizaciones por método
5. Integración con Google BigQuery y hoja de ruta cloud

1. Arquitectura y tecnologías utilizadas

La plataforma sigue una arquitectura cliente-servidor clásica con separación completa entre frontend y backend. Ambas capas se ejecutan de forma independiente y se comunican exclusivamente mediante una API REST.

1.1 Backend — Python / FastAPI

El servidor está construido sobre **FastAPI**, un framework web moderno para Python basado en estándares OpenAPI y tipado estático (Pydantic). Se sirve mediante **Uvicorn**, un servidor ASGI de alto rendimiento. La elección de FastAPI frente a alternativas como Flask o Django REST se justifica por su validación automática de esquemas, documentación interactiva generada automáticamente (*/docs*) y rendimiento superior en operaciones asíncronas.

Librería	Versión mín.	Uso en el proyecto
fastapi	0.111.0	Framework web principal — enrutamiento, validación, OpenAPI
uvicorn[standard]	0.29.0	Servidor ASGI — sirve la app FastAPI
python-multipart	0.0.9	Parsing de formularios y subida de archivos
pandas	2.0.0	Manipulación y transformación del DataFrame en todos los pasos
numpy	1.24.0	Operaciones numéricas; soporte a scikit-learn
scikit-learn	1.3.0	Clustering (K-Means, DBSCAN), métricas (Silhouette, etc.)
openpyxl	3.1.0	Lectura de archivos Excel (.xlsx, .xls)
google-cloud-bigquery	3.11.0	Ejecución de consultas SQL sobre BigQuery
google-auth	2.22.0	Autenticación con cuenta de servicio de Google Cloud
db-dtypes	1.1.1	Conversión de tipos de datos propios de BigQuery a pandas
pydantic	(con FastAPI)	Validación y serialización de modelos de entrada/salida

Estructura del backend

El código se organiza en cuatro capas bien definidas:

- **routers/** — Capa de transporte HTTP. Define los endpoints REST y delega la lógica a los servicios. Hay cuatro routers: `upload`, `mapping`, `cleaning`, `segmentation`.
- **services/** — Lógica de negocio: parseado de archivos, mapeo de columnas, limpieza, agregación RFM, algoritmos de segmentación y conector BigQuery.
- **models/** — Modelos Pydantic para validar las peticiones y estructurar las respuestas.
- **core/** — Sesión en memoria (`session_store.py`) y clases de excepción personalizadas.

- **utils/** — Utilidades de clustering (selección de k óptimo) y exportación CSV.

Gestión de sesiones

Cada proceso de segmentación genera un **session_id** único (UUID). La sesión almacena en memoria los DataFrames intermedios: datos crudos, datos mapeados, datos limpios y resultados de segmentación. Este diseño permite que el frontend avance paso a paso sin re-enviar los datos en cada petición.

- El almacén de sesiones es un diccionario en memoria (diseño de prototipo). En producción se sustituiría por Redis u otro almacén distribuido.

1.2 Frontend — React + Vite

La interfaz es una **Single Page Application (SPA)** construida con React 18. El scaffolding y el servidor de desarrollo utilizan **Vite**, que ofrece HMR (Hot Module Replacement) instantáneo y builds de producción optimizados.

Librería / Herramienta	Versión	Función
react + react-dom	^18.3.1	Framework UI — componentes, hooks, ciclo de vida
vite + @vitejs/plugin-react	^5.3.1	Build tool — dev server, HMR, bundle optimizado
zustand	^4.5.2	Gestión de estado global (wizard, sesión, idioma)
axios	^1.7.2	Cliente HTTP para llamadas a la API backend
recharts	^2.15.4	Librería de gráficos SVG (barras, dispersión, Pareto, pie)

Arquitectura del estado — Zustand

El estado de la aplicación se divide en tres stores independientes:

- **wizardStore** — Estado central del flujo: paso actual, método elegido, resultados de cada etapa. Cuando el usuario retrocede un paso, el store limpia automáticamente los datos posteriores para forzar la re-ejecución.
- **sessionStore** — Persiste el session_id en localStorage para sobrevivir recargas de página.
- **uiStore** — Estado de la interfaz: idioma activo (inglés / español).

Internacionalización (i18n)

Toda la interfaz es bilingüe (inglés / español de España). Las cadenas de texto se almacenan en un objeto de traducciones (translations.js) con las claves de ambos idiomas. El hook useT() recupera el objeto del idioma activo desde uiStore. El botón de cambio de idioma está situado en la cabecera, junto al botón de "Empezar de nuevo".

Flujo de la aplicación — 6 pasos

Paso	Nombre	Descripción
1	Método	Selección del algoritmo de segmentación
2	Datos	Carga del dataset (archivo o BigQuery)
3	Mapeo	Asignación de columnas del archivo a las métricas del sistema
4	Limpieza	Filtros de calidad: nulos, negativos, duplicados
5	Configuración	Parámetros específicos del método elegido
6	Resultados	Tabla de segmentos, descarga CSV, dashboard interactivo

2. Métodos de segmentación implementados

La plataforma implementa cuatro métodos de segmentación de clientes, cada uno con un flujo de configuración, parámetros y dashboard propios. El usuario elige el método en el Paso 1 y a partir de ese momento toda la experiencia (etiquetas, opciones disponibles, gráficos) se adapta automáticamente.

2.1 RFM Quintiles

Concepto: Divide a los clientes en cinco grupos (quintiles 1–5) según cada una de las tres métricas RFM y los asigna a segmentos nombrados mediante reglas de expresiones regulares.

Métricas:

- **R — Recency:** días desde la última compra. Puntuación 5 = cliente muy reciente, 1 = lleva mucho tiempo sin comprar.
- **F — Frequency:** número de pedidos únicos. Puntuación 5 = compra con mucha frecuencia.
- **M — Monetary:** gasto total acumulado. Puntuación 5 = cliente de alto valor.

Proceso:

- Se calculan los quintiles de cada métrica sobre el conjunto completo de clientes.
- Se concatenan las tres puntuaciones en un código RFM (p. ej. "543").
- Se recorre la lista de patrones de segmento en orden. El primer patrón que coincide con el código RFM del cliente determina su segmento.
- Los patrones usan sintaxis de expresión regular acotada: [4-5] significa puntuación 4 ó 5, [1] significa exactamente 1.

Segmentos predeterminados (15): Champions, Loyal Customers, Potential Loyalists, Recent Customers, Occasional Customers, Potential Customers, Economic Loyalists, Risky Customers, Nearly Lost, Need Attention, Average Customers, Non Active, Sleeping, New Customers, Lost.

Parámetros configurables en el Paso 5:

- El usuario puede **editar el nombre** de cualquier segmento predeterminado.
- Puede **modificar el patrón** R/F/M de cada segmento directamente en la tabla.
- Puede **desactivar segmentos** que no le resulten relevantes (checkbox).
- Puede **añadir segmentos nuevos** definiendo un nombre y sus rangos R/F/M.
- El orden importa: se aplica el primer patrón que coincida (first-match wins).

2.2 RFM K-Means / DBSCAN

Concepto: En lugar de reglas fijas, agrupa clientes mediante algoritmos de clustering no supervisado sobre el espacio tridimensional RFM. Los segmentos emergen de los propios datos, sin etiquetas predefinidas.

Pre-procesamiento: Las tres métricas RFM se normalizan con `StandardScaler` para que ninguna dimensión domine por escala.

Algoritmo K-Means:

- Se prueban todos los valores de `k` en el rango `[k_min, k_max]` configurado.
- Para cada `k` se calculan cuatro métricas de calidad: **WCSS** (inercia), **Silhouette Score**, **Davies-Bouldin Index** y **Calinski-Harabasz Score**.
- El método de selección automática de `k` puede ser: **combined** (40% Silhouette + 30% Davies-Bouldin invertido + 30% Calinski-Harabasz), o cualquiera de ellos de forma individual.
- Se ofrece una vista previa del gráfico de codo (elbow chart) antes de ejecutar la segmentación completa, permitiendo al usuario forzar manualmente un `k` concreto.

Algoritmo DBSCAN:

- **ϵ (Epsilon):** radio de vecindad. Controla qué tan cerca deben estar dos puntos para considerarse vecinos.
- **min_samples:** número mínimo de vecinos para que un punto sea núcleo. Los puntos sin suficientes vecinos se etiquetan como "Noise".
- DBSCAN encuentra clústeres de forma arbitraria y no requiere especificar `k`. Especialmente útil cuando los segmentos no son esféricos.

Parámetros configurables:

- Selección del algoritmo: K-Means o DBSCAN.
- Rango `k_min / k_max` (K-Means).
- Método de selección automática de `k`: `combined`, `silhouette`, `davies_bouldin`, `elbow`.
- Override manual de `k` tras ver el gráfico de codo.
- ϵ y `min_samples` (DBSCAN).

2.3 Análisis ABC (Pareto)

Concepto: Basado en el principio de Pareto (80/20), clasifica a los clientes en tres categorías según su contribución acumulada a los ingresos totales. Es el método más sencillo y rápido, ideal para una primera priorización.

Proceso:

- Se calcula el gasto total por cliente (suma de `order_total`).
- Se ordenan los clientes de mayor a menor gasto.
- Se calcula el porcentaje acumulado de ingresos que representa cada cliente.

- Se asigna clase A a los primeros clientes que acumulan hasta el umbral_A, clase B a los siguientes hasta el umbral_B, y clase C al resto.

Umbral predeterminados: A = 80%, B = 95% (es decir, B cubre el 80–95% acumulado y C el 95–100%).

Parámetros configurables:

- Umbral A (slider 50%–90%): porcentaje de ingresos acumulado para la clase A.
- Umbral B (slider: umbral_A+1% a 99%): límite superior de la clase B.

2.4 LRFMS — Segmentación por series temporales

Referencia académica: Wang, S., Sun, L. & Yu, Y. (2024). *Scientific Reports*, 14, 17491. Este es el método más avanzado y experimental de los cuatro implementados.

Concepto: A diferencia de los métodos estáticos (RFM snapshot), LRFMS divide el historial de pedidos en N intervalos temporales iguales y calcula cinco métricas por cada (cliente, intervalo):

- **L (Length):** amplitud del comportamiento del cliente en el intervalo. Medida como el número de días entre la primera y la última compra del intervalo.
- **R' (Recency-prime):** tiempo medio entre las últimas P transacciones y el final del intervalo. Captura la recencia reciente, no solo la última compra.
- **F (Frequency):** número de pedidos únicos en el intervalo.
- **M (Monetary):** gasto total en el intervalo.
- **S (Satisfaction):** métrica de satisfacción por intervalo (ver más abajo).

Con este enfoque se construye una **matriz de series temporales** (clientes × intervalos × métricas). Esta matriz se normaliza y se clusteriza con K-Means para identificar patrones de evolución en el tiempo.

La métrica S (Satisfacción) — Implementación actual

La implementación actual de S deriva la puntuación de satisfacción a partir de la **tasa de devoluciones por intervalo**. El fundamento es que un cliente que devuelve muchos artículos está, presumiblemente, insatisfecho. No se requieren datos de encuestas.

Tasa de devoluciones	Puntuación S	Interpretación
0%	9.0	Perfecto — sin devoluciones
1 – 2%	8.5	Excelente
3 – 5%	7.5 – 8.0	Muy bueno
6 – 10%	6.0 – 6.5	Bueno
11 – 20%	4.0	Problemas evidentes

> 20%	2.0	Problemas graves
-------	-----	------------------

Para activar S basada en devoluciones, el usuario debe marcar la opción "Tratar filas con cantidad negativa como eventos de devolución" en el **Paso 3 (Mapeo)**. Al hacerlo:

- Las filas con cantidad negativa se preservan durante la limpieza (no se eliminan).
- El sistema genera dos DataFrames paralelos: uno de pedidos normales y otro que incluye los eventos de devolución.
- La opción "Eliminar valores negativos" en el Paso 4 queda desactivada visualmente.
- En el Paso 5, se muestra un banner verde confirmando que las devoluciones están activas.

Peso de S

El parámetro **s_weight** controla la influencia de S en el clustering:

- **s_weight = 0** — S queda completamente excluida. El clustering usa solo L, R', F, M.
- **s_weight = 1.0** — S tiene el mismo peso que las demás métricas.
- **s_weight > 1.0** — S se amplifica, aumentando su influencia relativa.

Trabajo en curso — S desde encuestas

■ **Línea de investigación activa:** Se está trabajando en la implementación de S a partir del sistema propuesto en el paper original, donde la satisfacción se extrae de tres encuestas vinculadas a cada pedido: **pre-entrega** (expectativas antes del envío), **entrega** (experiencia en la recepción) y **post-entrega** (satisfacción tras el uso). Cuando el dataset incluya estas valoraciones mapeadas a la columna "satisfaction", el sistema calculará S como la media ponderada de las tres encuestas en lugar de derivarla de la tasa de devoluciones.

Demás parámetros configurables de LRFMS:

- **n_intervals (2–12):** número de intervalos temporales en que se divide el historial. Un valor mayor captura más granularidad pero requiere más datos por cliente.
- **p_value (1–10):** número de transacciones recientes que se promedian para calcular R'. Un p mayor suaviza la recencia.
- **k_min / k_max:** rango de número de clústeres a evaluar.
- **Método de selección de k:** igual que en RFM K-Means (combined, silhouette, davies_bouldin, elbow).

3. Tratamiento de valores nulos y decisiones de calidad de datos

La calidad del dato es una etapa crítica antes de cualquier proceso de segmentación. Un cliente mal identificado o un pedido sin importe introduce ruido que distorsiona los resultados.

3.1 Reporte de calidad pre-limpieza

Inmediatamente después de subir el archivo, el sistema genera un **informe de calidad** que se muestra en el Paso 4:

- Tabla con el número de valores nulos y valores negativos por columna.
- Número total de filas duplicadas.
- Indicadores visuales: fondo amarillo si el problema afecta entre 0 y 10% de las filas, fondo rojo si supera el 10%.

3.2 Opciones de limpieza

El usuario puede activar o desactivar tres filtros de forma independiente:

- **Eliminar filas con valores nulos** en columnas requeridas (`customer_id`, `order_id`, `order_date`, `order_total`).
- **Eliminar filas con valores numéricos negativos** (`order_total < 0`). Se desactiva automáticamente si el usuario ha configurado filas negativas como eventos de devolución en LRFMS.
- **Eliminar filas duplicadas** (misma combinación `customer_id` + `order_id` + `order_date`).

3.3 Decisión de diseño sobre `customer_id` nulo

■ **Decisión de diseño:** Los valores nulos en la columna **`customer_id`** se eliminan siempre, independientemente de que el filtro "Eliminar nulos" esté activado o no. El razonamiento detrás de esta decisión es el siguiente: un `customer_id` nulo significa que no se puede identificar a quién pertenece ese pedido. Si se agruparan todos estos pedidos bajo un "cliente nulo", se estaría creando una entidad artificial que concentraría transacciones de múltiples clientes desconocidos, sesgando gravemente las métricas RFM de ese "cliente". Introducir este tipo de dato artificial en el modelo de segmentación produciría un segmento espurio sin ningún valor de negocio. Se prefiere perder esas filas a contaminar el análisis.

3.4 Eliminación de `order_total` cero/negativo

Además de los filtros opcionales, el sistema aplica siempre una regla fija: las filas cuyo **`order_total`** ≤ 0 se eliminan del DataFrame de pedidos normales, ya que no aportan valor monetario y distorsionarían

las métricas M y F. La única excepción es cuando el usuario activa el modo de devoluciones en LRFMS, caso en que las filas con `order_total` negativo se preservan en un DataFrame paralelo (*orders_with_returns_df*) exclusivamente para el cálculo de S.

3.5 Deduplicación

La deduplicación se realiza sobre el DataFrame **crudo** (antes de cualquier transformación), lo que garantiza que el conteo de duplicados sea exacto y no esté afectado por el procesamiento previo de columnas. Una vez eliminados los duplicados, se aplica el resto de filtros sobre el conjunto ya limpio.

4. Dashboards y visualizaciones por método

Cada método de segmentación tiene un **dashboard diferente** diseñado para destacar la información más relevante de ese análisis concreto. El dashboard se abre como una capa modal sobre la página de resultados y utiliza **Recharts** para todas las visualizaciones.

Gráficos comunes a todos los métodos

Los siguientes gráficos aparecen en todos los dashboards:

- **Gráfico de pastel** — Distribución de clientes por segmento (%). Incluye etiqueta con nombre del segmento y porcentaje.
- **Gráfico de barras — Clientes por segmento** — Número absoluto de clientes, ordenados de mayor a menor.
- **Gráfico de barras — % Clientes vs. % Ingresos** — Comparación lado a lado de la cuota de clientes y la cuota de ingresos de cada segmento.
- **Top 10 clientes por segmento** — Tabla con los 10 mayores clientes de cada segmento (seleccionable con botones), ordenados por valor monetario.

4.1 Dashboard RFM Quintiles — gráficos exclusivos

- **Perfil RFM medio normalizado (barras agrupadas)** — Muestra la recencia, frecuencia y valor monetario promedio de cada segmento, normalizados entre 0 y 100% para hacer comparables las tres métricas. Permite identificar qué dimensión caracteriza a cada segmento.

4.2 Dashboard RFM K-Means — gráficos exclusivos

- **Gráfico de burbuja — Recencia vs. Monetario** — Cada burbuja representa un segmento/clúster. El eje X es la recencia media, el eje Y el valor monetario medio, y el tamaño de la burbuja es proporcional al número de clientes. Revela el posicionamiento relativo de los clústeres en el espacio RFM.
- **Perfil RFM medio normalizado (barras agrupadas)** — Igual que en Quintiles (ver arriba).
- **Tabla de perfil de clúster con renombrado** — Permite renombrar los clústeres (p. ej., de "Cluster 0" a "High Value") y muestra las estadísticas completas por clúster: media, desviación típica, mínimo y máximo de R, F y M. El renombrado se refleja en el archivo CSV descargado.

4.3 Dashboard ABC Analysis — gráficos exclusivos

- **Curva de Pareto ABC (gráfico compuesto, ancho completo)** — Es el gráfico central del dashboard ABC. Muestra la curva de ingresos acumulados frente al porcentaje de clientes ordenados por gasto descendente. Incluye áreas coloreadas para las zonas A (azul), B (naranja) y C (rosa), líneas de referencia verticales en los umbrales A y B, y líneas horizontales en los

porcentajes de ingresos correspondientes.

4.4 Dashboard LRFMS — gráficos exclusivos

- **Tabla de perfil de clúster LRFMS con renombrado** — Igual que en K-Means pero con las métricas propias de LRFMS: R' (recencia-prime), Frecuencia y Monetario. Las etiquetas se adaptan automáticamente al método.

■ **Nota sobre el renombrado de clústeres:** En los métodos K-Means y LRFMS, los clústeres se generan con nombres genéricos ("Cluster 0", "Cluster 1"...). La sección de perfil del dashboard permite al usuario asignarles nombres de negocio significativos. Al pulsar "Guardar nombres", el sistema actualiza tanto la vista del dashboard como el archivo CSV que se puede descargar.

5. Integración con Google BigQuery y hoja de ruta cloud

La plataforma ofrece actualmente dos vías de entrada de datos. La primera, carga de archivos locales (CSV/Excel). La segunda, consulta directa a **Google BigQuery**, lo que permite trabajar con datasets de cualquier tamaño sin necesidad de exportar ficheros.

5.1 Implementación actual de la entrada desde BigQuery

Mecanismo de autenticación:

La autenticación se realiza mediante una **cuenta de servicio de Google Cloud**. El usuario sube el archivo JSON de la clave de la cuenta de servicio directamente en la interfaz (mediante drag & drop o selector de archivos). Alternativamente, puede pegar el contenido JSON manualmente.

La cuenta de servicio debe tener asignados los roles:

- **BigQuery Data Viewer** — para leer datos de las tablas.
- **BigQuery Job User** — para crear y ejecutar jobs de consulta. **Este rol es imprescindible** ya que BigQuery procesa las consultas como jobs asíncronos incluso para SELECT simples; sin él se produce un error 403 de scope insuficiente.

Flujo técnico:

- El JSON de la clave se envía al backend vía POST /upload/bigquery.
- El servicio `bigquery_connector.py` valida el JSON (campos obligatorios: `type`, `project_id`, `private_key`, `client_email`).
- Se construyen las credenciales con el scope `https://www.googleapis.com/auth/bigquery` (scope completo, no readonly).
- Se ejecuta la consulta SQL y el resultado se convierte en un DataFrame de pandas.
- A partir de ahí, el flujo es idéntico al de un archivo subido localmente.

Importaciones opcionales (lazy imports):

El módulo `google-cloud-bigquery` se importa dentro de un bloque `try/except` al arrancar el backend. Si el paquete no está instalado, el conector devuelve un error 501 (Not Implemented) en lugar de impedir el arranque del servidor. El script de inicio (`{MONO("start.sh")}`) instala automáticamente todas las dependencias del `{MONO("requirements.txt")}` antes de arrancar el servidor.

5.2 Hoja de ruta — Trabajo en curso

■ **Funcionalidades en desarrollo activo:**

- **Escritura de resultados en BigQuery:** Una vez completada la segmentación, poder escribir directamente los resultados (customer_id + segmento + métricas) en una tabla de BigQuery destino, eliminando la necesidad de descargar y re-subir el CSV a otros sistemas.
- **Ejecución en Google Cloud:** El objetivo es mover todos los procesos que actualmente se ejecutan en local —especialmente los algoritmos de clustering y la agregación RFM de grandes volúmenes— a infraestructura de Google Cloud. Las opciones evaluadas incluyen Cloud Run (contenedores serverless), Cloud Functions (ejecución de funciones individuales) y Vertex AI (para los pasos de ML más intensivos como K-Means y LRFMS). Esto permitiría procesar datasets de millones de clientes sin limitaciones de memoria local.
- **Orquestación de pipelines:** Integración con Cloud Composer (Apache Airflow) para programar ejecuciones periódicas automáticas de la segmentación (p. ej., mensual) sobre datos frescos de BigQuery.

5.3 Arquitectura objetivo (cloud-native)

Componente actual	→	Equivalente cloud
Servidor Uvicorn local (FastAPI)	→	Cloud Run (contenedor Docker)
Archivos CSV en disco local	→	Cloud Storage (GCS)
Sesiones en memoria (dict)	→	Cloud Memorystore (Redis)
Clustering local (scikit-learn)	→	Vertex AI Training Jobs
Resultados en CSV descargable	→	Escritura directa en BigQuery
Ejecución manual	→	Cloud Scheduler + Cloud Composer

Documento generado automáticamente a partir del código fuente del repositorio. Para información actualizada consultar el propio código en `product/`.